



Instituto Tecnológico de Costa Rica
Centro Académico de Alajuela
Ingeniería en computación

Proyecto 2: Simulador P2P

Una implementación de un simulador P2P

IC-6600 Principios de Sistemas Operativos
I semestre, 2026

Estudiantes:

Angie Esquivel López
Alejandro Quesada Calderón
Jose Adrián Herrera Salas

Prof. Eddy Ramírez Jiménez

Alajuela, Junio, 2026

Índice

1. Introducción	3
2. Marco teórico	4
2.1. Arquitectura Peer-to-Peer (P2P)	4
2.2. Sockets de Red y Comunicación Interprocesos	4
2.3. Hilos y Concurrencia	5
2.4. Sincronización y Condiciones de Carrera	5
2.5. Tablas Hash en la Gestión de Archivos	6
3. Descripción de la solución	7
3.1. Arquitectura general	7
3.2. Protocolo de comunicación	7
3.3. Servidor P2P	8
3.4. Cliente y control por consola	9
3.5. Transferencia y reconstrucción de archivos	9
3.6. Búsqueda distribuida	10
3.7. Hash e identidad de archivos	10
3.8. Concurrencia y sincronización	10
3.9. Pruebas y validación	11
4. Conclusiones	12
5. Descripción de la experiencia	13
6. Aprendizajes personales	14
6.1. Jose Herrera	14
6.2. Alejandro Quesada	14
6.3. Angie Esquivel	14

1. Introducción

Las redes *Peer-to-Peer* (P2P) constituyen un modelo de comunicación distribuida en el que cada nodo puede actuar simultáneamente como cliente y servidor, permitiendo compartir recursos de manera directa sin depender completamente de un servidor central. Este tipo de arquitectura es ampliamente utilizado en aplicaciones de distribución de archivos debido a su escalabilidad, tolerancia a fallos y aprovechamiento de los recursos disponibles en la red.

El presente proyecto consiste en el desarrollo de un simulador de una red P2P utilizando programación con *sockets*, concurrencia mediante hilos y protocolos de comunicación definidos por el equipo. El sistema implementa un servidor encargado de almacenar los metadatos de los archivos compartidos y múltiples clientes capaces de registrar sus archivos, realizar búsquedas, solicitar archivos a otros clientes y reconstruirlos a partir de múltiples transferencias concurrentes.

Como parte de la solución, se implementa una función de *hash* para identificar de manera única los archivos independientemente de su nombre, además de un mecanismo de búsqueda distribuida que permite a los clientes localizar archivos incluso cuando el servidor central no se encuentra disponible. Para ello, los clientes mantienen una lista de vecinos conocidos y propagan consultas utilizando un identificador único y un contador de saltos (TTL), evitando ciclos y reduciendo el tráfico innecesario en la red.

El desarrollo de este proyecto permite aplicar diversos conceptos fundamentales de los Sistemas Operativos, entre ellos la comunicación mediante sockets, el manejo de entrada y salida, la programación concurrente, la sincronización entre procesos, la gestión de protocolos de red y el funcionamiento de sistemas distribuidos. Asimismo, ofrece una aproximación práctica al diseño de aplicaciones capaces de compartir información de forma eficiente y tolerante a fallos.

2. Marco teórico

2.1. Arquitectura Peer-to-Peer (P2P)

En contraste con el modelo tradicional cliente-servidor (donde los roles de proveedor y consumidor de recursos están rígidamente separados), las redes *Peer-to-Peer* (P2P) adoptan un enfoque descentralizado. En un entorno P2P no se distinguen clientes y servidores dedicados, en su lugar, todos los nodos del sistema se consideran pares (*peers*). [1]

Bajo esta arquitectura, un nodo individual puede actuar simultáneamente como cliente (solicitando servicios o datos a otros componentes de la red) y como servidor (proveyendo recursos a los nodos que lo requieran). El descubrimiento de servicios en estos entornos distribuidos puede gestionarse mediante un servidor central de registro o de forma completamente distribuida, donde los nodos cooperan directamente para localizar los recursos compartidos.

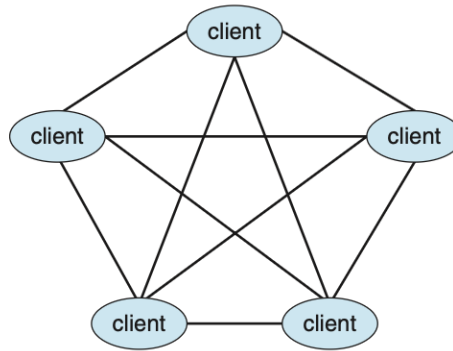


Figure 1.23 Peer-to-peer system with no centralized service.

Figura 1: Peer-to-Peer [1]

2.2. Sockets de Red y Comunicación Interprocesos

La comunicación entre diferentes sistemas a través de una red se fundamenta conceptualmente en el uso de *sockets*. Un *socket* se define formalmente como un punto final (*endpoint*) para la comunicación interprocesos a través de una infraestructura de red.

Desde la perspectiva del sistema operativo, la identidad y ubicación de un *socket* se establece mediante la concatenación de dos elementos fundamentales: una dirección IP y un número de puerto. Esta combinación permite que el sistema operativo encamine los flujos de datos entrantes y salientes de manera unívoca hacia el proceso de aplicación correspondiente, permitiendo la entrega estructurada de bytes entre el cliente y el servidor en la arquitectura simulada.[1]

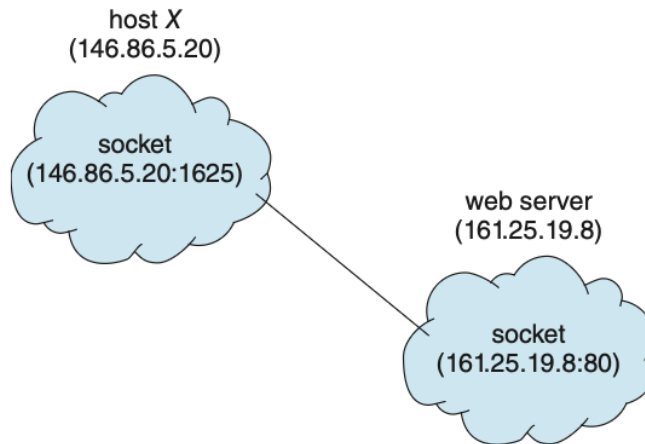


Figure 3.26 Communication using sockets.

Figura 2: Imagen que ilustra las partes de un *socket* de red [1]

2.3. Hilos y Concurrency

Un hilo (*thread*) constituye la unidad básica de utilización de la CPU dentro de un sistema operativo moderno. Un hilo comparte con sus hilos hermanos pertenecientes al mismo proceso recursos críticos como la sección de código, la sección de datos y los recursos del sistema operativo (tales como archivos abiertos o conexiones de red). Sin embargo, mantiene de forma independiente su propio identificador de hilo, un contador de programa (PC), un conjunto de registros y una pila.[1]

La adopción de una arquitectura multihilo es esencial en aplicaciones de red concurrentes. Permite que un proceso continúe su ejecución y responda a las interacciones del usuario en primer plano (interfaz de consola), mientras delega tareas intensivas de entrada/salida (I/O) (como la transferencia o recepción de fragmentos de archivos en el sistema P2P) a hilos independientes en segundo plano.[1]

2.4. Sincronización y Condiciones de Carrera

Cuando un sistema operativo o una aplicación ejecuta múltiples hilos de manera concurrente, surge la necesidad de coordinar el acceso a los recursos compartidos. Una condición de carrera (*race condition*) ocurre cuando varios hilos acceden y manipulan simultáneamente los mismos datos, de modo que el resultado final de la ejecución depende estrictamente del orden particular en el que se llevan a cabo los accesos.[1]

Para prevenir la corrupción de datos y garantizar la consistencia en estructuras compartidas (tales como las listas de vecinos o los registros de identificadores de consultas procesadas), es indispensable implementar herramientas de sincronización. Estas herramientas aseguran la exclusión mutua, permitiendo que únicamente un hilo a la vez pueda operar dentro de su sección crítica.[1]

2.5. Tablas Hash en la Gestión de Archivos

En la implementación de sistemas de archivos y directorios, la eficiencia en la localización de recursos es un factor crítico de rendimiento. El uso de una estructura de datos de tipo tabla hash permite optimizar sustancialmente los tiempos de búsqueda en comparación con las búsquedas lineales tradicionales.[1]

Una tabla hash utiliza una función de dispersión para transformar una clave o propiedad del archivo (como sus metadatos o contenido unívoco) en un índice numérico dentro de un arreglo. Teóricamente, esto reduce el tiempo necesario para verificar la existencia o ubicación de un archivo a un tiempo constante promedio ($O(1)$), lo que resulta ideal para comprobar de forma unívoca la identidad de los datos compartidos en la red distribuida, independientemente de las modificaciones en sus nombres comerciales.[1]

3. Descripción de la solución

La solución desarrollada consiste en un simulador P2P escrito en C que utiliza sockets TCP, hilos POSIX y un protocolo propio de mensajes binarios para registrar archivos, buscar metadatos y transferir rangos de bytes entre clientes. La implementación sigue la especificación del proyecto: existe un servidor central para el registro inicial y la búsqueda centralizada, pero los clientes también pueden buscar de forma distribuida entre vecinos cuando se requiere tolerancia ante la ausencia del servidor. La identidad lógica de un archivo se define por el par (S, H) , donde S corresponde al tamaño en bytes y H al hash calculado sobre el contenido. De esta forma, dos archivos con nombres distintos pero con los mismos bytes se tratan como el mismo recurso compartido.

3.1. Arquitectura general

El sistema se divide en cinco áreas principales: componentes comunes, servidor, cliente, transferencia de archivos y búsqueda distribuida. Los componentes comunes se encuentran en `common/` e incluyen la definición del protocolo, la función de hash y las funciones auxiliares para sockets. El servidor se encuentra en `server/` y mantiene el registro global de pares y archivos. El cliente se encuentra en `client/` y controla el arranque, el escaneo de la carpeta compartida y la interfaz de consola. La transferencia de datos vive en `transfer/` y permite enviar y recibir rangos específicos de un archivo. Finalmente, `search/` implementa la lista de vecinos, la propagación de consultas, el control de TTL, la deduplicación de consultas y la agregación de resultados.

El flujo básico de ejecución es el siguiente:

1. Se inicia el servidor con un puerto de escucha.
2. Cada cliente se inicia con la IP y puerto del servidor, su propio puerto de datos y la ruta de la carpeta compartida.
3. El cliente escanea recursivamente su carpeta, calcula metadatos de cada archivo y se registra ante el servidor.
4. El servidor guarda los metadatos y devuelve una lista de vecinos recientes para alimentar la búsqueda distribuida.
5. El usuario ejecuta comandos de búsqueda y solicitud desde la consola del cliente.
6. Si se solicita un archivo, el cliente divide el rango total entre los pares disponibles, descarga los segmentos en paralelo y reconstruye el archivo en su propia carpeta compartida.

3.2. Protocolo de comunicación

Todos los módulos se comunican mediante un protocolo definido en el archivo de contrato común del proyecto. Cada mensaje TCP se envía como una trama con prefijo de longitud usando las funciones de red comunes. Dentro de esa trama, el primer campo es un encabezado

común que contiene el código de operación, la versión del protocolo y el tamaño del payload. Los campos numéricos multibyte se serializan en orden de red para mantener compatibilidad entre sistemas.

Los códigos de operación principales son:

- **REGISTER_REQ** y **REGISTER_RESP**, usados por el cliente para publicar sus archivos y recibir vecinos iniciales.
- **FIND_REQ** y **FIND_RESP**, usados para búsquedas por nombre y para consultas de identidad (S,H).
- **TRANSFER_REQ** y **TRANSFER_DATA**, usados para pedir y enviar rangos de bytes de archivos.
- **QUERY_FLOOD** y **QUERY_RESULT**, usados por la búsqueda distribuida.
- **ERROR**, usado para rechazar mensajes mal formados o versiones incompatibles.

El protocolo se mantiene congelado para evitar incompatibilidades entre los tres dominios de trabajo. En vez de agregar un nuevo mensaje para solicitudes por identidad, el cliente reutiliza **FIND_REQ** con términos como "S H", "S:H" o "S=<S> H=<H>". El servidor interpreta ese término como búsqueda exacta por tamaño y hash, y responde con todos los pares que poseen el archivo.

3.3. Servidor P2P

El servidor central atiende conexiones TCP entrantes y crea un hilo por cada cliente aceptado. Su estado principal es un registro en memoria protegido por `pthread_mutex_t`. Este registro contiene las entradas de pares activos, el puerto de datos anunciado por cada uno, la marca de tiempo de la última actualización y la lista de archivos compartidos por ese par.

Cuando recibe un **REGISTER_REQ**, el servidor actualiza o inserta la entrada del par con los metadatos enviados. Cada archivo se almacena con nombre, tamaño, hash, IP del dueño y puerto de datos. Como respuesta, el servidor devuelve hasta **P2P_MAX_NEIGHBORS** vecinos recientes, excluyendo al cliente que acaba de registrarse. Esa respuesta permite que los clientes formen su topología inicial para la búsqueda distribuida.

Cuando recibe un **FIND_REQ**, el servidor puede operar en dos modos. Si el término contiene texto ordinario, realiza una búsqueda por subcadena sobre los nombres de archivo registrados. Si el término tiene formato de identidad, busca coincidencias exactas por **size_bytes** y **hash**. En ambos casos retorna una lista de metadatos con los campos necesarios para que el cliente pueda mostrar resultados y posteriormente solicitar el archivo. El servidor también valida versiones, tamaños de payload y códigos de operación desconocidos, devolviendo **ERROR** cuando corresponde en vez de terminar el proceso.

3.4. Cliente y control por consola

El cliente concentra la interacción con el usuario. Al iniciar, recibe por línea de comandos la dirección del servidor, el puerto del servidor, su puerto de datos y la carpeta compartida. También acepta parámetros opcionales para el TTL de búsqueda distribuida y la ventana de espera de respuestas. Antes de entrar a la consola, el cliente ignora SIGPIPE, escanea la carpeta compartida, calcula hash y tamaño de cada archivo, inicia el listener de datos y se registra ante el servidor.

La consola implementa cuatro comandos principales:

- `find -s <nombre>`: consulta directamente al servidor central.
- `find -d <nombre>`: ejecuta búsqueda distribuida entre vecinos.
- `find <nombre>`: intenta primero la búsqueda centralizada y, si el servidor falla o no devuelve resultados, cae a búsqueda distribuida.
- `request <S> <H>`: solicita el archivo identificado por tamaño y hash.

Los resultados de búsqueda se imprimen con el tamaño, hash, IP, puerto y nombre del archivo. Además, el cliente conserva la última lista de resultados para que un `request` posterior pueda usar esos metadatos. Antes de descargar, el cliente refresca la lista de pares mediante una búsqueda de identidad en el servidor; si esa búsqueda no produce pares, usa los resultados que ya estaban en caché. Esta decisión permite que el comando funcione tanto después de `find -s` como después de `find -d`.

3.5. Transferencia y reconstrucción de archivos

Cada cliente levanta un listener en su puerto de datos. Ese listener es único para mensajes entre pares y despacha tanto solicitudes de transferencia como mensajes de búsqueda distribuida. Cuando llega un `TRANSFER_REQ`, el cliente receptor interpreta el hash, tamaño y rango solicitado, localiza en su carpeta compartida un archivo que coincida con (S,H) y envía los bytes pedidos mediante una o más tramas `TRANSFER_DATA`.

En el lado solicitante, `transfer_request` recibe la lista de pares que poseen el mismo archivo. Si hay varios pares disponibles, divide el rango `[0,S-1]` en partes equilibradas; el último segmento recibe el remanente cuando la división no es exacta. Para cada segmento se crea un hilo que abre una conexión TCP con el par asignado, envía la solicitud de rango y escribe los datos recibidos en la posición correcta del archivo parcial usando `pwrite`. Cuando todos los hilos terminan correctamente, el archivo parcial se sincroniza, se cierra y se renombra al nombre final dentro de la carpeta compartida del solicitante.

El nombre final conserva el nombre original reportado por la búsqueda. Por ejemplo, si el archivo remoto se llama `apple.pdf`, la descarga se incorpora como `apple.pdf`. La implementación toma únicamente el nombre base para evitar que un metadato remoto con separadores de ruta pueda escribir fuera de la carpeta compartida. Si no existe un nombre disponible, se utiliza un nombre determinista basado en S y H como respaldo.

El manejo de desconexiones y carpetas removidas en caliente se realiza mediante validaciones antes de abrir archivos y verificaciones de errores en operaciones de I/O. Si la carpeta

compartida de origen o destino desaparece, el proceso registra una advertencia y continúa ejecutándose cuando es posible, cumpliendo el requisito de que ni el cliente ni el servidor se caigan por un retiro de dispositivo.

3.6. Búsqueda distribuida

La búsqueda distribuida reduce la dependencia del servidor central. Cada cliente mantiene una lista de vecinos inicializada con los pares recientes que devuelve el servidor durante el registro. Además, puede actualizarla al recibir consultas de otros pares. La lista está protegida por mutex para permitir acceso desde el hilo de la consola y desde los hilos que atienden conexiones entrantes.

Cuando se ejecuta `find -d`, el cliente crea un `query_msg_t` con el término de búsqueda, un identificador pseudoaleatorio de consulta, su IP y puerto de origen, y un TTL configurable. La consulta se envía por TCP a todos los vecinos conocidos. Al recibirla, un par verifica si ya procesó ese `query_id`; si ya lo vio, descarta el mensaje para evitar ciclos. Si es nuevo, revisa su metadata local previamente calculada al iniciar el cliente, responde directamente al originante cuando encuentra coincidencias y reenvía la consulta con TTL decrementado si todavía quedan saltos disponibles.

Las respuestas `QUERY_RESULT` se agregan en un acumulador global protegido por mutex durante una ventana configurable, por defecto de dos segundos. Al finalizar la espera, el cliente recolecta los resultados y los muestra con el mismo formato usado para la búsqueda centralizada. Esta equivalencia de formato permite que `request <S> <H>` use resultados de búsqueda distribuida sin requerir una interfaz separada.

3.7. Hash e identidad de archivos

La función de hash implementada es una variante FNV-1a de 64 bits aplicada al contenido del archivo. El escáner del cliente calcula el hash y el tamaño al registrar archivos, y esos dos valores se usan como identidad estable del contenido. Esta decisión cumple el requisito de reconocer archivos equivalentes aunque cambie su nombre: el nombre sirve para búsqueda y presentación, mientras que el par `(S,H)` sirve para decidir qué archivo debe transferirse.

Durante la transferencia, el emisor vuelve a localizar un archivo por tamaño y hash en su carpeta compartida antes de enviar bytes. El receptor también valida que cada trama `TRANSFER_DATA` pertenezca al hash, tamaño y rango esperado. Si un segmento llega duplicado, fuera de orden o con metadatos inconsistentes, la recepción se considera inválida y la descarga parcial se descarta.

3.8. Concurrencia y sincronización

La solución utiliza varios niveles de concurrencia. El servidor acepta conexiones y atiende cada solicitud en un hilo independiente, mientras el registro compartido se protege con mutex. En el cliente, el hilo principal permanece en la consola; un listener separado acepta mensajes entre pares, y cada conexión entrante se procesa en su propio hilo. Para una descarga, el cliente crea un hilo por segmento para poder solicitar diferentes rangos del mismo archivo a diferentes pares en paralelo.

La búsqueda distribuida también depende de sincronización. La lista de vecinos, el registro de consultas vistas y el acumulador de respuestas están protegidos por mutex para evitar condiciones de carrera. En particular, el acumulador de respuestas se limpia entre búsquedas sin destruir el mutex que puede estar usando el hilo listener, lo que evita interferencias entre la consola y la recepción de resultados asíncronos.

3.9. Pruebas y validación

La implementación se acompaña de pruebas unitarias e integraciones locales. Las pruebas unitarias cubren la función de hash, el framing de red, el contrato de protocolo, el registro del servidor, el manejador de consultas, el emisor de transferencias y el receptor de transferencias. En el receptor se valida tanto la reconstrucción de bytes como la preservación del nombre original descargado.

Las pruebas de integración levantan procesos reales de servidor y clientes, ejecutan comandos por consola y verifican los resultados observables. Cubren registro y búsqueda centralizada, solicitud posterior a búsqueda centralizada, búsqueda por identidad, solicitud sin búsqueda previa, búsqueda distribuida, solicitud posterior a búsqueda distribuida, fallback de `find` plano a búsqueda distribuida, retiro de carpeta compartida durante una solicitud y un escenario grande opcional con tres clientes y un archivo `.mkv` de más de 500 MiB. En conjunto, estas pruebas verifican que los módulos de servidor, cliente, transferencia y búsqueda distribuida trabajen integrados sobre el mismo protocolo.

4. Conclusiones

1. La separación del sistema en módulos permitió que el servidor, el cliente, la transferencia de archivos y la búsqueda distribuida avanzaran de forma independiente sin perder un contrato común. El archivo de protocolo fue especialmente importante porque todos los componentes dependen de los mismos códigos de operación, estructuras y reglas de serialización.
2. El uso de TCP simplificó la entrega confiable de mensajes y segmentos de archivos, pero también obligó a manejar con cuidado los casos de lectura parcial, escritura parcial, desconexiones y errores de red. Las funciones comunes de red ayudaron a que esas validaciones no se repitieran de manera distinta en cada módulo.
3. La identidad de archivos mediante el par (S, H) resultó útil para reconocer contenido equivalente aunque el nombre cambie. Esta decisión también hizo posible refrescar los pares disponibles antes de una descarga y dividir el archivo entre varios clientes que comparten el mismo contenido.
4. La descarga segmentada mostró por qué la concurrencia debe diseñarse con una estrategia clara. Dividir rangos entre pares puede mejorar el uso de la red, pero requiere validar metadatos, escribir cada segmento en su posición correcta y responder de forma controlada ante fallos o retiros de la carpeta compartida.
5. La búsqueda distribuida complementa al servidor central y reduce su papel como punto único de falla. Sin embargo, para que sea viable en una red local se necesitan controles como TTL, deduplicación de consultas y una ventana de agregación de respuestas.
6. Las pruebas unitarias e integraciones fueron necesarias para comprobar el comportamiento real del simulador. En particular, los escenarios con múltiples clientes, búsqueda centralizada, búsqueda distribuida, transferencia y hot-unplug permitieron encontrar problemas que no eran visibles al compilar cada módulo por separado.

5. Descripción de la experiencia

6. Aprendizajes personales

6.1. Jose Herrera

Desde el rol de servidor y protocolo, el principal aprendizaje fue comprender que una arquitectura distribuida depende mucho de la claridad del contrato entre módulos. Definir los mensajes, los campos obligatorios, la identidad de los archivos, el orden de bytes y las respuestas de error no fue solo una tarea de diseño, sino una base para que el cliente y la búsqueda distribuida pudieran integrarse sin ambigüedades. Además, el trabajo con el registro de pares mostró la importancia de proteger estructuras compartidas con mutex y de diseñar un servidor que continúe funcionando aunque un cliente envíe mensajes inválidos o se desconecte inesperadamente.

6.2. Alejandro Quesada

Desde el rol de cliente, red y transferencia, el aprendizaje más importante fue pasar de una interfaz de consola sencilla a un flujo completo de comunicación entre procesos. El cliente no solo debía escanear archivos y enviar comandos, sino también registrarse, interpretar respuestas del servidor, iniciar un listener, coordinar descargas concurrentes, reconstruir archivos por rangos y mantenerse estable ante fallos como carpetas removidas o pares no disponibles. La implementación de `request <S> <H>` permitió ver con claridad la relación entre metadatos, búsqueda previa, refresco de pares y ensamblaje final del archivo descargado.

6.3. Angie Esquivel

Desde el rol de búsqueda distribuida y documentación, el aprendizaje principal fue que un sistema P2P necesita controlar la propagación para ser útil. La lista de vecinos, los identificadores de consulta, la deduplicación, el TTL y la agregación de respuestas trabajan juntos para permitir que una búsqueda encuentre archivos sin saturar la red ni bloquear la consola del cliente. También quedó claro que la documentación debe mantenerse cerca de la implementación: explicar las decisiones de diseño, las pruebas y las limitaciones ayuda a que el proyecto sea entendible para el equipo y reproducible en el ambiente de evaluación.

Referencias

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*. Wiley, 10th ed., 2018.